gemstone-js README

gemstone-js

`gemstone-js` is the TypeScript bridge to GemStone/S described in `../plan.js.txt`.

This initial slice locks in the public async API, OOP encoding helpers, runtime dispatch layer, test mock, session/pool scaffolding, and a separate `@gemstone-js/native` napi-rs starter. The live GCI implementation is in the native package; the TypeScript package can be tested locally with a mock runtime.

Current Status

- Async-first API: `Session.connect()`, `connectFromEnv()`, `with()`, `withEnv()`, `execute()`, `perform()`, `performWith()`, `performValueWith()`, `performObjectWith()`, `bulkPerformOop()`/`bulkPerformValue()` and mixed-call `bulkPerformCallsOop()`/`bulkPerformCallsValue()` with `performMany*()`/`performCalls*()` aliases, plus `bulkPerformWith()` and mixed-call `bulkPerformCallsWithOop()` variants that batch repeated sends after normal JavaScript argument marshalling, retained object batch helpers such as `bulkPerformObjects()` and `bulkPerformCallsObjectsWith()`, typed/managed execute helpers, `withTransaction()` with sync or async callbacks, framework-neutral `RequestScope`/`TransactionScope` helpers, and `AsyncDisposable` support.
- High-level argument marshalling through `performWith()` and `ManagedOop.send()`: strings become GemStone strings, numbers become SmallIntegers or Floats, bigints become SmallIntegers, booleans/null become immediate OOPs, arrays become GemStone `Array` objects, plain objects become `StringKeyValueDictionary`, and managed handles pass their retained OOP.
- Opt-in value converters mirror gemstone-py's explicit converter registry: `ValueConverter`, `ValueConverterRegistry`, `dateAsIsoStringConverter()`, and `scalarValueConverterRegistry()` let sessions marshal richer JavaScript values such as `Date` through the standard `performWith()`, array, and dictionary paths. `objectToDictionaryArgument()`/ `classInstanceToDictionaryArgument()` convert class instances into explicit dictionary payloads before persistence, mirroring gemstone-py's `dataclass_to_dict()` pattern. See `docs/object-mapping.md` for the supported mapping layers and the current non-ORM boundary.
- `mappedObject()` provides an opt-in transparent mapping layer around `TypedOop<T>`: async property-style methods, explicit `$send*()` controls, setters, object/raw selector modes, relationship handles, dictionary snapshot fields, and bounded snapshots. The object-mapping guide now includes a layer-selection table, selector configuration examples, relationship mapping, root/global mapping, lifetime and transaction rules, migration guidance from raw `TypedOop<T>` sends to generated refs, and a production checklist. The planned connector-inspired track extends this with a mapping manifest schema, generated `*Ref` classes, repository helpers, and Explorer/VS Code mapping views.
- `transparentObject()` is the higher-transparency object mapping layer: `await booking.status` sends a GemStone selector, accessor properties remain callable as methods, runtime assignment queues setter sends for `$flush()`, `$assign()` gives typed write batching, optional per-proxy caching supports `$refresh()`/`$clearCache()`, and `TransparentObjectMapper` reuses proxies by session/OOP for request-scoped identity.
- `smalltalkBridge()` provides a gemstone-py-style dynamic Smalltalk bridge for exploratory and tool code: globals resolve lazily from properties, JavaScript-friendly names such as `new_` and `at_put_` map to `new:` and `at:put:`, selector accessors are awaitable or callable, selector object results can go straight to `transparent()`/`transparentWith()` proxies, and `$send*()` plus `$transparent()` keep exact-selector and retained-handle paths available.
- `Session.classRef()` gives an explicit typed class handle for class-side sends, object-returning sends, allocation, and wrapping returned OOPs without hiding async remote calls behind JavaScript property access.

Class names use the same simple GemStone global-name validation policy as source-rendered helpers.

- `GbsSessionParameters`/`GbsSession` provide a small GemStone-Pharo-Bridge compatibility facade for MagLev branch examples: `login()`, `userGlobals`, `bridgeRoot`, `commit()`, `commitTransactionOrSignalConflict()`, retry-count commits with a work callback, and `disconnect()`.
- Runtime adapters: Node (`@gemstone-js/native`), Deno FFI starter, Bun FFI starter, and a mock runtime for tests.
- Runtime library discovery follows `libPath`, `GS_LIB_PATH`, `GS_LIB`, then `GEMSTONE/lib` for the Deno and Bun FFI adapters.
- Deno and Bun FFI adapters now marshal pointer-buffer calls for `perform()`, `fetchBytes()`, float conversion, `GciErr`, and dictionary lookups through shared typed array helpers.
- OOP helpers ported from `gemstone-rs/crates/gemstone-gci`.
- Low-level allocation/fetch helpers: `newOop()`, `fetchClass()`, `fetchSize()`, `fetchBytes()`, `arrayToOop()`, `arrayOopToValues()`, raw `arrayOopToOops()`/`arrayOops()`, retained object `arrayOopToObjects()`/`arrayObjects()`, direct array `arraySize()`/`arrayIsEmpty()`/`arrayAt*()`/`arrayFirst*()`/`arrayLast*()` plus bounded `arrayPage*()`/`arrayTake*()`, batch `arrayPick*()`/`arraySetAll*()` helpers, and retained `array()` wrappers. Array value readback accepts optional `maxDepth` and `maxItems`/`maxTotalItems` bounds to guard recursive or unexpectedly large arrays; raw OOP readback accepts a `maxItems` bound.
- Dictionary/global helpers: `dictionaryToOop()`, `dictionaryOopToObject()`, `dictionaryValues()`, `dictionaryKeys()`, `dictionarySize()`, `dictionaryIsEmpty()`, `dictionaryEntries()`, `dictionaryItems()`, raw `dictionaryEntriesOop()`/`dictionaryItemsOop()`, value-list `dictionaryValueList()`, raw `dictionaryValueOops()` shared `KeyedReadbackOptions`/`DictionaryReadbackOptions` `maxEntries` bounds for dictionary key/value/item readback, direct dictionary `get`/`set`/`replace`/`remove`/`clear` plus `has`/`pick`/`require` helpers for raw, value, object, and nested-dictionary entries, `strDictGet()`, `strDictSet()`, `globalGet()`/`globalGetValue()`, `globalGetObject()`, `globalHas()`/`globalHasAll()`, `globalKeys()`, `globalPick()`, raw `globalPickOop()`, nullable object/dictionary `globalPickObject()`/`globalPickDict()`, `globalEntries()`, raw `globalEntriesOop()`, `globalValues()`, `globalItems()`, raw `globalValuesOop()`/`globalItemsOop()`, bounded `KeyedReadbackOptions` `maxEntries` options for global key/value/item readback, `globalSize()`/`globalIsEmpty()`, `globalSet()`/`globalSetValue` and `globalSetAll()`/`globalSetAllValue()`, raw `globalSetOop()`/`globalSetAllOop()` plus object-named `globalSetObject()`/`globalSetAllObject()` aliases, required global accessors including object alias `globalRequire()` and `globalRequireAll*()` bulk variants, dictionary helpers `globalGetDict()`, `globalGetDictObject()`, `globalSetDict()`/`globalSetAllDict()`, `globalRequireDict()`/`globalRequireDictObject()` and `globalRemove()`/`globalDelete()` plus bulk `globalRemoveAll()`/`globalDeleteAll()`.
- `GsDict` wraps GemStone `StringKeyValueDictionary` objects with `get()`/`getValue()`, `getObject()`, `set()`/`setValue()` and `setAll()`/`setAllValue()`, raw `setOop()`/`setAllOop()` plus object-named `setObject()`/`setAllObject()` aliases, `replaceAll()`/`replaceAllValue()`, raw `replaceAllOop()`, object-named `replaceAllObject()`, dictionary `replaceAllDict()`, `clear()`, `remove()`/`delete()`, `has()`, `size()`, `isEmpty()`, `keys()`, `values()`, raw `valuesOop()`, `items()`, raw `itemsOop()`, `entries()`, bounded `maxEntries` readback options, `toObject()`, raw `entriesOop()`, `pick()`, raw `pickOop()`, nullable object/dictionary `pickObject()`/`pickDict()`, bulk `hasAll()` and `removeAll()`/`deleteAll()`, required raw/value/object accessors plus object alias `require()` and `requireAll*()` bulk variants, nested dictionary helpers `getDict()`/`setDict()`/`setAllDict()`/`requireDict()`/`requireAllDict()`, explicit send helpers, and inspection helpers.
- `PersistentRoot` now has value helpers (`getValue()`, `setValue()`, `setAllValue()`, `getDict()`, `getDictObject()`, `setDict()`/`setAllDict()`), raw `setAll()`, explicit `setOop()`/`setAllOop()` and object-named `setObject()`/`setAllObject()` aliases, `getObject()`, `remove()`/`delete()`, `removeAll()`/`deleteAll()`, `has()`, `hasAll()`, `keys()`, `pick()`, raw `pickOop()`, nullable object/dictionary `pickObject()`/`pickDict()`, `entries()`, raw `entriesOop()`, `values()`, raw `valuesOop()`, `items()`, raw `itemsOop()`, `size()`/`isEmpty()`, and required raw/value/object/dictionary access plus `requireDictObject()` for plain bounded dictionary snapshots and `requireAll*()` bulk variants built on the session marshalling layer. Static constructors expose `UserGlobals`, `Globals`, `Published`, and `SessionMethods` roots using the same names as gemstone-py.
- `OrderedCollection` wraps live GemStone ordered sequences with `session.orderedCollection()` creation, `wrapOrderedCollection()` for existing OOPs, append/extend/remove/clear operations, nullable zero-based indexed access including negative indexes, first/last/pop/shift helpers, value/raw OOP/object readback, async iteration, reverse value iteration, and direct send/inspect helpers.
- `GStore` provides a session-bound, named JSON key/value store under `UserGlobals.GStoreRoot`, with async transaction callbacks, read-only snapshots, bounded `GStore.list()`/`read()`/transaction snapshot options,

static and instance existence checks, delete/remove helpers, and commit-conflict retry for conflict-like commit failures.

- Module-style migration helpers provide dependency-safe `planUpgrade()`/`planDowngrade()`, live `upgrade()`/`downgrade()`, current version/status reads, checksum validation, recorded dry-runs, and a `gemstone-js-migrations` CLI. Metadata and advisory locks live in `UserGlobals` as JSON strings under `GemstoneJsMigrations` and `GemstoneJsMigrationsLock`; see `docs/migrations.md`.
- Transaction retry helpers provide `runTransactionWithRetry()` and `retryingTransaction()` for replaying work after commit conflicts, plus `CommitConflictError`, `TransactionRetry`, structured conflict diagnostics, formatting helpers for logging or CI output, and `nestedTransaction()` for GemStone nested transaction blocks.
- Benchmark tooling can generate compact reports through `gemstone-js-benchmarks` using an offline `gci` suite or opt-in live persistence suites, then validate saved reports, compare baseline/candidate artifacts with global, suite, and operation regression thresholds, select or directly compare against matching committed baselines by metadata, and register/prune baseline manifests through `gemstone-js-benchmark-validate`, `gemstone-js-benchmark-compare`, `gemstone-js-benchmark-baselines`, and `gemstone-js-benchmark-register`; see `docs/benchmarks.md`.
- Reduced-conflict wrappers expose GemStone `RcCounter`, `RcKeyValueDictionary`, and `RcQueue` through `RcCounter`, `RcKeyValueDictionary`, and `RcQueue`, with gemstone-py-compatible aliases `RCCounter`, `RCHash`, and `RCQueue`. The wrappers cover class-side creation, `session.rcCounter()`/`rcKeyValueDictionary()`/`rcQueue()` factories, value/raw/object sends, counter increments and guarded decrements, dictionary get/set/batch-set/remove/enumeration/rebuild helpers, and queue push/batch-push/pop/peek/indexed reads.
- `bootstrapGemStone()` and `gemstone-js-bootstrap` audit or initialize the GemStone-side roots used by the persistence helpers: `GStoreRoot`, `GSQueryRoot`, and `GemstoneJsBootstrapVersion`.
- `gemstone-js-doctor` checks local runtime, GemStone environment variables, GCI library discovery, optional native-package importability, native worker method-surface compatibility, and optional live login/eval without printing configured secrets; see `docs/doctor.md`.
- Session pool release is reset-aware: dirty sessions are aborted before reuse, failed resets are discarded, waiters are served with replacement sessions, and close rejects pending acquires. Explicit validation queries run without needing a separate interval option, `warm()` is idempotent for target capacity, custom health checks and max-age/max-use recycling match production provider patterns, `snapshot()` and `eventListener` expose provider-style operational events, and `withSession()` wraps sync or async acquire/use/release callback flows.
- `RequestScope`, `TransactionScope`, `requestFailed()`, `sessionScope()`, and `withSessionScope()` provide the framework-neutral request/session lifecycle layer used by web adapters: lazy session acquisition, commit-on-success, abort-on-error/status, clean pool release, and owned-session logout.
- Express, Fastify, Fetch API, and Hono adapters now delegate request teardown through the shared scope layer, expose the active `gemstoneScope`, and accept `transactionPolicy`/`serverErrorStatus` options while preserving the existing default abort-on-4xx behavior. Runnable adapter examples live in `examples/web-express.ts`, `examples/web-fastify.ts`, `examples/web-fetch.ts`, and `examples/web-hono.ts`, with route-handler exports in `examples/web-route-handler.ts`; the dependency-free `examples/explorer.ts` browser example adds status, OOP inspection, roots/globals, workspace eval, class browsing, and codegen preview. See `docs/framework-adapters.md` and `docs/examples-guide.md`.
- `gemstone-js-examples` lists packaged examples, filters them by kind, prints JSON, guided plans, and runnable command lists for tooling, and can show an example file directly from the installed npm package. The packaged catalog includes quickstart, data helpers, query helpers, bulk selector sends, migrations, ObjectLog, codegen, web adapter examples, and the local explorer; see `docs/examples-guide.md`.
- `gemstone-js-compare` prints the local JS/Python, Rust/Python, and combined ecosystem comparison, including scorecards, gap rows, next batches, batch totals, and schema-versioned JSON output for release notes, CI, or planning tools.
- Result marshalling now converts GemStone `String` and `Symbol` objects back into JavaScript strings via `fetchString()` and class detection. Float OOPs are converted when the runtime reports that `GciOopToFlt_` succeeded.
- `Session.inspect()`, bounded recursive `dump()`, direct `printString()`, `describeClass()`, class-ref `describe()`, and retained handle `inspect()`/`dump()`/`printString()` helpers return typed `oop`, `class`, `classOop`, `printString`, size/byte-size, class hierarchy, slot, indexed-field, superclass, class instance-variable, and instance-count metadata for quick debugging of raw object handles and classes.
- `ObjectLog` wraps GemStone `ObjectLogEntry` with async add, bounded read, options-driven `{ level,`

order, maxEntries } fetches, tail/latest reads, server-side level filtering, latest-by-level helpers, count/presence checks, summary/format helpers, size, level-scoped clear, and single/bulk delete helpers. Batched entry fetches preserve real zero-based log indexes and use the same escaped row parser shape as gemstone-py.

- GSCollection.search() unwraps result arrays into typed handles, GSCollection.searchOop() returns raw handles, all()/allOop() read the collection as object handles, value helpers such as allValues(), pageValues(), searchValues(), limitValues(), firstValue(), iterValues(), and item value helpers marshal result objects directly, page()/pageOop() fetch bounded array pages, at()/itemAt() and raw atOop()/itemAtOop() read nullable 1-based indexed items, firstItem()/lastItem() and raw firstItemOop()/lastItemOop() read collection endpoints without predicate scans, add()/addAll() and raw addOop()/addAllOop() append values, includes()/contains() and raw includesOop()/containsOop() check collection membership, remove()/delete() and raw removeOop()/deleteOop() remove members, removeAll()/removeAllOop() remove batches, and replaceAll()/replaceAllOop() plus clear() manage whole collections, first()/firstOop() return nullable first matches without materializing result arrays, find()/findOop()/findValue() alias the first-match helpers, limit()/take() fetch bounded predicate matches, size()/isEmpty() expose collection metadata, count() increments a counter without materializing selected matches, exists() and any()/anyMatch()/none() early-exit with detect:ifNone:, and GSCollection.iter() fetches collection chunks while yielding individual objects. Query comparison operators are validated before Smalltalk source is emitted. Equality-index helpers are available through both explicit createEqualityIndexOn()/removeEqualityIndexOn() and higher-level createIndex()/removeIndex() calls. Source-rendering helpers validate collection names, persistent-root names, and SymbolDictionary entry names before emitting Smalltalk; see docs/naming.md for the shared policy.
- Pool, observability hooks, persistent-root, query, codegen, and Express, Fastify, and Hono adapters.
- Codegen helpers render wrappers that share the same JavaScript argument marshalling as hand-written calls and can return marshalled values, raw OOPs, or retained typed object handles. Generated selectors are checked for keyword shape and argument arity before source is emitted; manifests also reject malformed function entries and duplicate exported wrapper names.
- Codegen manifests have a published JSON Schema at schemas/codegen-manifest.schema.json, can include typed signatures/imports, and can be produced from decorated source with npm run codegen:scan --. The scanner uses the TypeScript parser for decorators, decorator aliases, namespace decorators from gemstone-js, overload signatures, multiline methods, generics, and default, namespace, and aliased typed imports, and infers raw OOP/object-returning wrappers from Oop and TypedOop<T> return annotations. Add --module to emit generated wrapper source directly from decorated classes.
- npm run codegen -- [--check] <manifest.json> [output.ts] renders wrapper modules from a JSON manifest and can verify checked-in generated files in CI; see examples/codegen.manifest.json and examples/codegen.generated.ts. Manifest imports support direct typeNames, default typeDefaultName, aliased typeSpecifiers, and namespace typeNamespaceName imports.
- Benchmark report and baseline manifest JSON Schemas are published at schemas/benchmark-report.schema.json and schemas/benchmark-baseline-manifest.schema.json.
- examples/booking.decorators.ts is a committed scanner fixture; CI verifies that it still renders examples/booking.decorators.generated.ts.
- npm run public-surface:check compares the src/index.ts export barrel against a committed TypeScript-compiler-derived API contract so accidental breaking public exports fail local and release checks.
- gemstone-js-api-contract --json imports the package entrypoint and verifies that runtime value exports, package metadata, schema exports, and CLI bin targets match the committed source contract, giving release smoke checks a package-installed API probe similar to gemstone-py's api_contract command.
- Session.performObjectWith() and sendValue() aliases make value, raw OOP, and retained object-returning sends explicit at both session and class-ref layers.
- InMemoryMetrics and InMemoryTracer make observability behavior easy to assert in tests and examples.

Local Smoke Test

```
npm run verify
```

Node 24 can execute the .ts tests directly using built-in type stripping. The local verify path checks both checked-in generated outputs, example syntax, the public API surface contract, the runtime package API contract, CLI bin/schema metadata, and the checksum helper self-test, then uses npm pack --dry-run with a disposable cache

to verify that the publishable tarball includes docs/examples while excluding tests and local build metadata. It also packs and extracts the tarball to verify installed API and bin wiring. `npm run live:check` is part of `verify`; it does not connect to GemStone, but guards the opt-in live test skip path and coverage shape.

For the beta install path, native backend choice, generated-wrapper workflow, installed-package proof, and support boundary, see `docs/beta.md`. The final beta release-candidate gate is `GS_RUN_LIVE=1 npm run release-candidate:check`.

Live GemStone checks are opt-in:

```
GS_RUN_LIVE=1 npm run test:live
GS_RUN_LIVE=1 GS_NATIVE_SESSION_WORKER=1 npm run test:live
npm run test:live:worker
```

The live smoke covers connect, execute, class-side sends, `performWith()`, string, float, nested array marshalling/value/raw/page readback, bounded dictionary readback, bounded global lookup/enumeration/nullable object/bulk required/raw set/removal helpers, `GsDict` metadata, value/raw enumeration, replace/clear, nullable object/dictionary pick, bulk required-read, and bulk removal helpers, and `PersistentRoot` value, dictionary, bounded key, size, pick, required-value, batch-value, bulk nullable object/dictionary pick, required-read, and bulk removal helpers, plus `GStore` JSON transaction round-trips. It also covers live query add/remove, `count()`, `exists()`, `first()`, `limit()`, larger bounded query result paths, and index create/remove when the backing collection supports GemStone index selectors, generated wrapper value/raw OOP/object-return calls against built-in classes, migration upgrade/downgrade/current-version round-trips with a dedicated metadata root and advisory lock, and real `SessionPool`, queued-acquire pressure, `withSessionScope()`, and Fetch, Express, Fastify, and Hono adapter commit/abort lifecycles through local framework fakes. It also runs a focused `GciSessionWorker` backend stress path covering concurrent JavaScript-side calls, fetch bytes, retained object export-set retain/release, failed send recovery, and logout shutdown.

The inspection helpers are also available from the command line. The command uses the same `GS_*` connection environment as `Session.configFromEnv()`:

```
npm run inspect -- --oop 123456789
npm run inspect -- --oop 123456789 --dump --depth 2
npm run inspect -- --class Booking --json
```

GemStone-side helper roots can be audited or initialized with:

```
npm run bootstrap -- --status
npm run bootstrap -- --dry-run
npm run bootstrap
```

Module-style migrations use the same connection environment:

```
npm run migrations -- status --manifest ./migrations.ts
npm run migrations -- upgrade --manifest ./migrations.ts --dry-run --record
```

Benchmark report generation and saved-report tooling are local-only. Select only `gci` for an offline run; live suites use the same `GS_*` connection environment as `Session.configFromEnv()`:

```
npm run benchmarks -- --suite gci --entries 100000 --json --output benchmark-report.json
npm run benchmark:validate -- benchmark-report.json
npm run benchmark:compare -- baseline.json candidate.json --max-regression-pct 10
npm run benchmark:baselines -- candidate.json --manifest .github/benchmarks/index.json
npm run benchmark:register -- candidate.json --manifest .github/benchmarks/index.json
```

`Session.configFromEnv()` prefers the canonical JavaScript names `GS_USERNAME`, `GS_PASSWORD`, `GS_HOST`, `GS_NETLDI`, and `GS_GEM_SERVICE`. It also accepts the Pharo bridge live-test aliases `GS_USER`, `GS_PASS`, `GS_NETLDI_HOST`, `GS_NETLDI_NAME_OR_PORT`, and `GS_SERVICE`, so the same shell can drive both projects. Tooling can call `sessionConfigFromEnv()` and `sessionEnvAliasConflicts()` to apply the same policy outside `Session`.

Node uses the raw `@gemstone-js/native Gci` binding by default. To run each session through the native package's dedicated worker-thread wrapper, set `GS_NATIVE_SESSION_WORKER=1` or pass `nativeSessionWorker: true`

to `Session.connect()`. Keep the raw backend when debugging low-level GCI behavior; use the worker backend when you want blocking native calls queued onto one OS thread per session. Worker dispatch is strict, so missing worker methods fail at worker creation instead of falling back to raw native module calls. If the installed native package does not export `createGciSessionWorker()`, update `@gemstone-js/native` or disable the worker switch.

Example

```
import { Session } from "gemstone-js";

await using session = await Session.connect({
  username: process.env.GS_USERNAME,
  password: process.env.GS_PASSWORD,
});

const oop = await session.execute("1 + 1");
console.log(oop);
```

The smallest durable dictionary example can use environment-based login/logout and read the dictionary back as a bounded JavaScript snapshot:

```
import { Session } from "gemstone-js";

const key = "MyTestDict";

await Session.withEnv(async (session) => {
  await session.globalSetDict(key, {
    name: "Tariq",
    amount: 100,
    currency: "GBP",
  });
  await session.commit();
});

const saved = await Session.withEnv((session) =>
  session.globalRequireDictObject(key, { maxEntries: 50 })
);

console.log(saved);
// { name: "Tariq", amount: 100n, currency: "GBP" }
```

Runtime Notes

The current comparison with `gemstone-py` is tracked in `docs/gemstone-js-vs-gemstone-py.md`, with implementation-level parity notes in `docs/gemstone-py-parity.md`. The Rust/Python ecosystem view is summarized in `docs/gemstone-rs-comparison.md` and is available from `gemstone-js-compare gemstone-rs --scorecard`. Machine-readable comparison output uses `schemas/comparison-report.schema.json`; pass `--output <path>` to save the selected JSON, Markdown, or human report for CI artifacts or release notes. Use `--scope beta` for the narrower beta-hardening estimate, or the default `--scope full` for the broader product-parity plan. CI can pin expected planning numbers with `--assert-total-batches`, `--assert-hours-min`, and `--assert-hours-max`, or enforce upper bounds with `--max-total-batches`, `--max-hours-min`, and `--max-hours-max`. Use `--quiet` for no output on passing guard checks.

Node uses `@gemstone-js/native`, which is scaffolded in `../gemstone-js-native` and is intended to expose napi-rs bindings over the shared `gemstone-gci` Rust crate.

Deno and Bun adapters declare the GCI symbols directly with their built-in FFI systems. Pointer-array and out-parameter calls are wired through typed arrays; `GciErr` is decoded into the same `GciErrorInfo` shape used by the Node adapter. Live-runtime hardening is still needed around platform coverage.